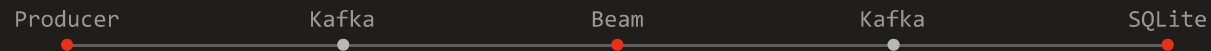


Métricas de pagos en tiempo real

Pipeline streaming con Apache Kafka, Apache Beam y SQLite.



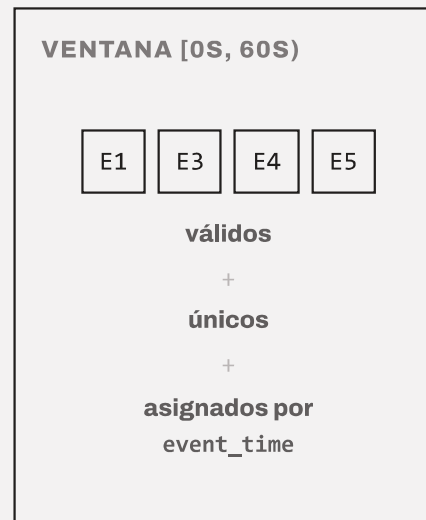
01 — PROBLEMA Y OBJETIVO

¿Cuántas transacciones CONFIRMED y cuánto PYG procesa cada comercio, por minuto?

EJEMPLO CONCEPTUAL — MERCHANT_003

EVENT_TIME	LLEGADA	ID	CONDICIÓN
10s	1º	E1	normal
45s	2º	E3	normal
45s	3º	E3	duplicado
30s	4º	E4	fuera de orden
20s	5º	E5	tardío
E5 llega tras el watermark, dentro de allowed lateness.			

→

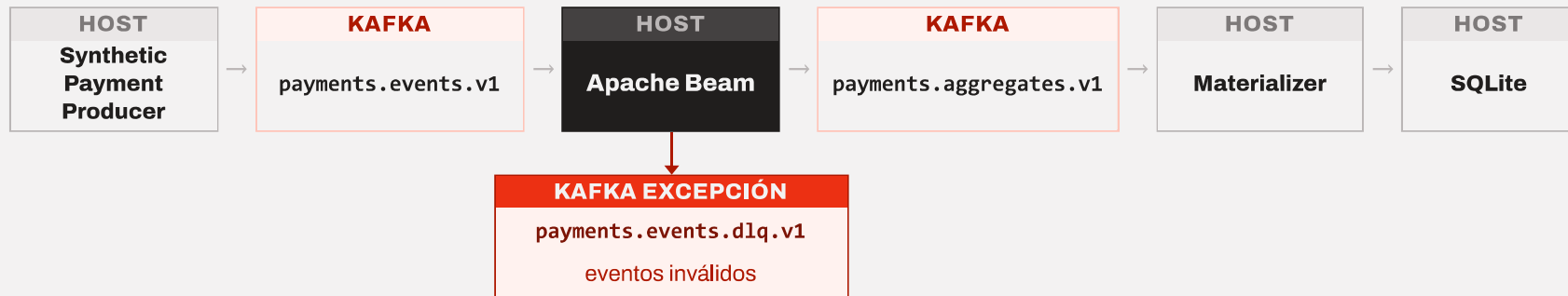


→

RESULTADO
merchant_id
window_start
transaction_count = N
total_amount_pyg = Σ amount_pyg

El resultado debe ubicar cada transacción válida en la ventana correcta y contarla una sola vez.

Arquitectura end-to-end



Kafka desacopla el flujo; Beam decide qué evento es válido, a qué ventana pertenece y qué aggregate emitir.

Tres campos, tres problemas distintos

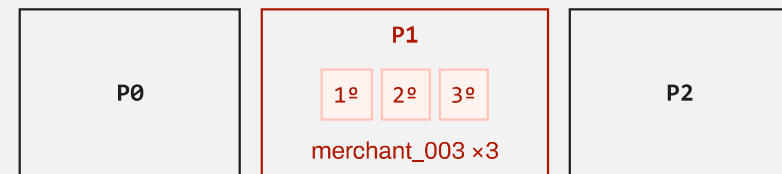
EVENTO SIMPLIFICADO

```
"event_id": "e_9f21",  
"merchant_id": "merchant_003",  
"event_time": "...T00:00:34Z",  
"status": "CONFIRMED", "amount_pyg": 400264
```

Kafka key = merchant_003



P1



event_id ----- → deduplicación

merchant_id ----- → Kafka key → partición → orden por comercio

event_time ----- → timestamp Beam → ventana

misma key → misma partición → orden relativo por comercio

input: 3 particiones · aggregates: 3 particiones · DLQ: 1 partición

Cada etapa responde una pregunta sobre el evento

KafkaO	Validate	CONFIRMED	Dedup	Window	CombinePerKey	Aggregate
¿qué entra?	→ ¿es válido?	→ ¿impacta la métrica?	→ ¿ya lo vimos?	→ ¿a qué minuto pertenece?	→ ¿resultado por comercio?	→ emite el resultado

AGGREGATE EMITIDO

merchant_id

window_start

transaction_count

total_amount_pyg

SECCIÓN

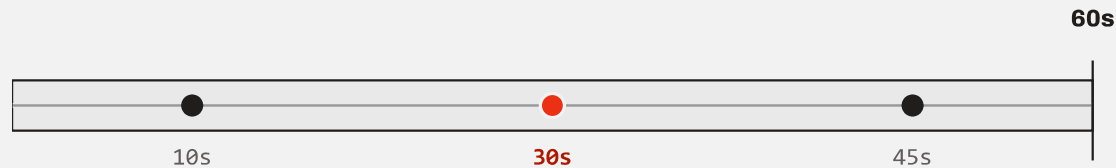
Procesamiento temporal

Event time, ventanas, triggers y lateness.

05 — EVENT TIME

Beam asigna ventanas por event_time, no por orden de llegada

EVENT TIME — VENTANA [0s, 60s)



SECUENCIA DE LLEGADA

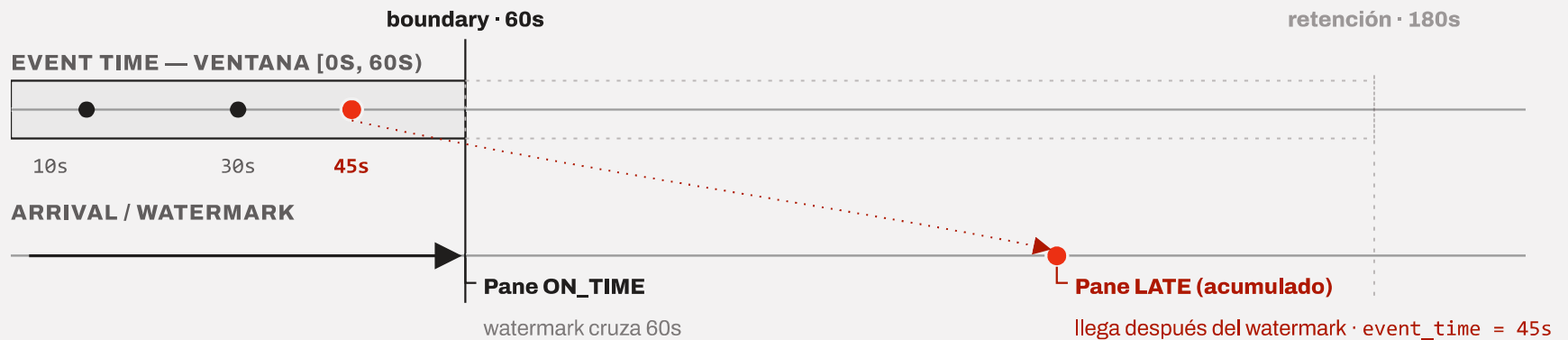


event_time = 30s llegó tercero, pero sigue perteneciendo a la ventana [0s, 60s).

OUT-OF-ORDER: el orden de llegada difiere del orden de event time.

Desorden no significa tardío: un evento puede llegar fuera de orden y seguir llegando a tiempo. Allowed lateness = 120s (contexto técnico, se explica en la próxima diapositiva).

Triggers, panes y lateness



`AfterWatermark(late=AfterCount(1))` modo **ACCUMULATING** — verificado con TestStream

07 — IDEMPOTENCIA

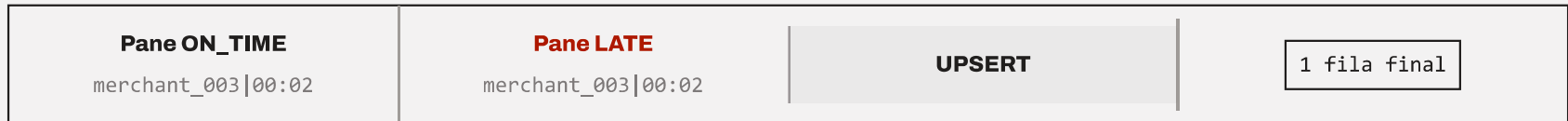
Dos mecanismos, no uno

ENTRADA DUPLICADA



Un evento duplicado no entra dos veces al agregado.

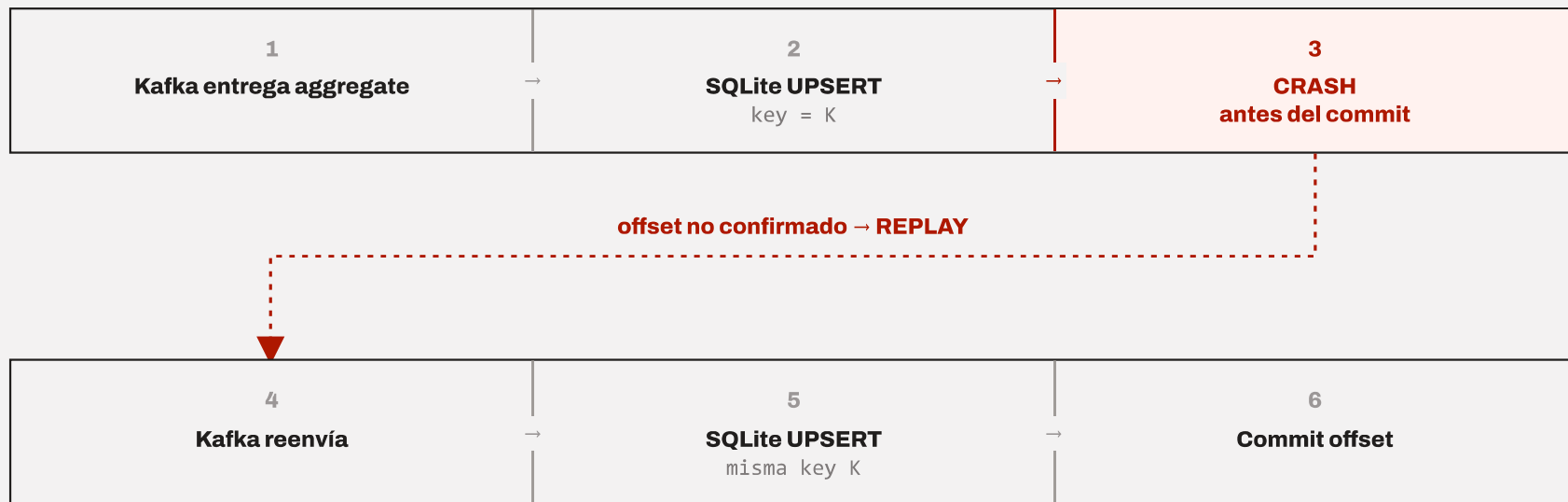
AGGREGATE MATERIALIZADO



Una nueva versión reemplaza el estado materializado; no se vuelve a sumar.

08 — ENTREGA

Qué pasa si el materializer falla a mitad de camino



KAFKA — offset no confirmado → puede reenviar → semántica **at-least-once**.

SQLITE — misma aggregate_key → UPSERT reemplaza → sink **idempotente**.

El proyecto no declara exactly-once.

SECCIÓN

Confiabilidad y evidencia

Resultado E2E, pruebas y límites.

Resultado E2E real

FLUJO PROCESADO

10 eventos publicados	→	1 evento inválido → DLQ	→	3 aggregates producidos	→	3 filas en SQLite
---------------------------------	---	-----------------------------------	---	-----------------------------------	---	-----------------------------

RESULTADO Y SALUD DEL PIPELINE

6 transacciones CONFIRMED únicas	1 527 772 PYG en transacciones CONFIRMED	lag 0 Apache Beam lag 0 Materializer
---	--	---

Estos valores corresponden a la corrida histórica registrada del Bloque 4 — el pipeline ya fue ejecutado y reconciliado end-to-end.

La demo reproduce el mismo flujo; con `--start-now` la distribución entre ventanas puede variar.

Qué prueba cada capa de tests

Unit / TestPipeline	TestStream	E2E real
Contratos, validación, agregación, dedup.	Event time, ventanas, desorden, lateness, panes.	Kafka → Beam → Kafka → SQLite

LÍMITES

DirectRunner local

Demo bounded (smoke E2E)

No exactly-once

La semántica temporal se prueba de forma determinista; la integración se verifica contra Kafka real.